

"Reporte de práctica Unidad II"

PRESENTADO POR:
DANIEL SALVADOR MEDINA COXCA
JESÚS ALFREDO MURRIETA MATUS

NO. DE CONTROL:
22TE0249
22TE0278

LICENCIATURA EN:
INGENIERÍA SISTEMAS COMPUTACIONALES

ASIGNATURA:
Inteligencia Artificial

Teziutlán, Puebla; octubre de 2025

"La Juventud de Hoy, Tecnología del Mañana"



INTRODUCCIÓN.....	3
Marco teórico	3
Qué es la representación del conocimiento en IA	3
Grafos.....	3
Librerías utilizadas en los ejercicios de clase	3
NetworkX	3
Matplotlib Pyplot.....	4
Pydub	4
SpeechRecognition.....	4
FastAPI	5
File y UploadFile (FastAPI)	5
HTTPException (FastAPI)	5
Form (FastAPI).....	5
Tkinter	5
Desarrollo	7
Representación del conocimiento con grafos.....	7
Chatbot.....	9
Reconocimiento de voz con servicios	13
Chatbot con audio y texto	16
Conclusiones	22

INTRODUCCIÓN.

La Inteligencia Artificial es un área que busca desarrollar sistemas capaces de imitar o reproducir ciertas capacidades humanas, como el aprendizaje, razonamiento y la interacción con su entorno. Uno de los aspectos fundamentales es la representación del conocimiento, pues busca almacenar, organizar y utilizar información de manera eficiente para la toma de decisiones. En este reporte de práctica se abordan conceptos vistos en clases como la representación del conocimiento mediante grafos, el diseño de chatbots, el reconocimiento de voz y la implementación que integren tanto audio como texto.

Marco teórico

Qué es la representación del conocimiento en IA

La representación del conocimiento es el proceso de modelar la información del mundo real de una forma que las computadoras puedan entender y manipular. Permite que los sistemas inteligentes razonen sobre los datos y encuentren soluciones. Existen diferentes enfoques de representación como reglas, marcos, ontologías y grafos.

Grafos

Los grafos son estructuras matemáticas formadas por nodos (o vértices) y aristas (conexiones). En IA se utilizan para representar relaciones entre conceptos. Por ejemplo, un grafo puede modelar redes sociales, mapas de rutas o conocimiento semántico (ontologías). La gran ventaja de los grafos es que permiten representar información compleja de manera flexible y visual.

Librerías utilizadas en los ejercicios de clase

NetworkX

Es una librería de Python diseñada para la creación, manipulación y estudio de grafos y redes complejas.

- Permite representar relaciones entre objetos usando nodos y aristas.
- Se utiliza en algoritmos de grafos (caminos más cortos, centralidad, cliques, etc.).
- Ejemplo: crear un grafo de amigos en una red social.

Matplotlib Pyplot

Matplotlib.pyplot es un módulo de la librería Matplotlib, usado para crear gráficos y visualizaciones en Python.

- Sirve para representar datos de manera gráfica (líneas, barras, dispersión, pastel, etc.).
- Se usa junto con librerías como NumPy o pandas.
- Ejemplo: graficar el crecimiento de usuarios de un chatbot en el tiempo.

Pydub

Es una librería de Python para procesamiento de audio.

Permite cortar, unir, convertir y aplicar efectos a archivos de audio (MP3, WAV, etc.).

Se usa cuando se necesita manipular audios en proyectos de IA, chatbots con voz o reconocimiento de audio.

Ejemplo: convertir un archivo WAV en MP3 para que lo procese un servicio de voz

SpeechRecognition

Es una librería de Python que permite convertir voz en texto usando motores de reconocimiento de voz.

Puede usar servicios como Google Speech API, Sphinx, Microsoft, IBM, entre otros.

Funciona con micrófono o archivos de audio.

Ejemplo: reconocer lo que dice el usuario en un chatbot por voz.

FastAPI

Es un framework web moderno y rápido para construir APIs con Python.

Basado en Python type hints y asíncrono (muy eficiente).

Ideal para aplicaciones de IA, chatbots y servicios en la nube.

Ventajas: velocidad, validación automática de datos y documentación integrada (Swagger).

File y UploadFile (FastAPI)

Son clases que proporciona FastAPI para manejar archivos en las APIs.

File: define parámetros para subir archivos en los endpoints.

UploadFile: representa un archivo subido, con propiedades como filename, content_type y métodos para leer/escribir.

Ejemplo: subir un archivo de audio al servidor para procesarlo con SpeechRecognition.

HTTPException (FastAPI)

Es una clase de FastAPI que se utiliza para manejar errores HTTP en las APIs.

Permite enviar mensajes de error al cliente con un código de estado (404, 400, 500, etc.).

Ejemplo: si el usuario sube un archivo de formato no válido, se devuelve un error 400 (Bad Request).

Form (FastAPI)

Es un tipo de entrada en FastAPI que permite recibir datos enviados mediante formularios HTML (application/x-www-form-urlencoded o multipart/form-data).

Se usa cuando el usuario llena un formulario en una página web y se envía a la API.

Ejemplo: recibir el nombre del usuario en un formulario junto con un archivo de audio.

Tkinter

tkinter es la librería estándar de Python para crear interfaces gráficas de usuario (GUI).

Viene incluida en la instalación de Python, por lo que no requiere instalación adicional.

Permite crear ventanas, botones, menús, cuadros de texto, etiquetas, entre otros elementos gráficos.

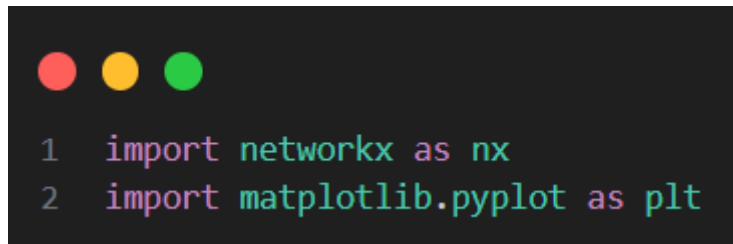
Es multiplataforma (funciona en Windows, macOS y Linux).

Sencilla de usar para aplicaciones pequeñas o educativas.

Desarrollo

Representación del conocimiento con grafos

Para esta primera parte de programación con IA en relación con los grafos se utilizaron librerías networkx y matplotlib.pyplot.



```
1 import networkx as nx
2 import matplotlib.pyplot as plt
```

Imagen 1.1 Importaciones de librería

Después de eso se creó la relación de los grafos utilizando la librería networkx, es importante colocar bien el nombre de los grafos pues puede generar un grafo erróneo al momento de mostrarlo, en este ejemplo se fue más a la parte de IA y sus ramas.



```
1 G = nx.DiGraph() #Grafo dirigido
2 G.add_edges_from([
3     ("IA", "Machine Learning"),
4     ("IA", "Deep Learning"),
5     ("Machine Learning", "Redes Neuronales"),
6     ("Machine Learning", "Arboles de Decisión"),
7     ("Deep Learning", "CNN"),
8     ("Deep Learning", "RNN")
9 ])
```

Imagen 1.2 Creación del grafo.

Para finalizar se implementó con el diseño del grafo utilizando la librería matplotlib.pyplot.

Utilizando colores como se muestra en el fragmento del código, y al final mostrar el grafo.

```

1 plt.figure(figsize=(8,6))
2 nx.draw(G, with_labels =True, node_color='lightblue',
3         edge_color='gray', node_size=300, font_size=10)
4 plt.show()
5

```

Imagen 1.2 Diseño del grafo

Al ejecutar el programa muestra el grafo terminado en una venta con sus relaciones.

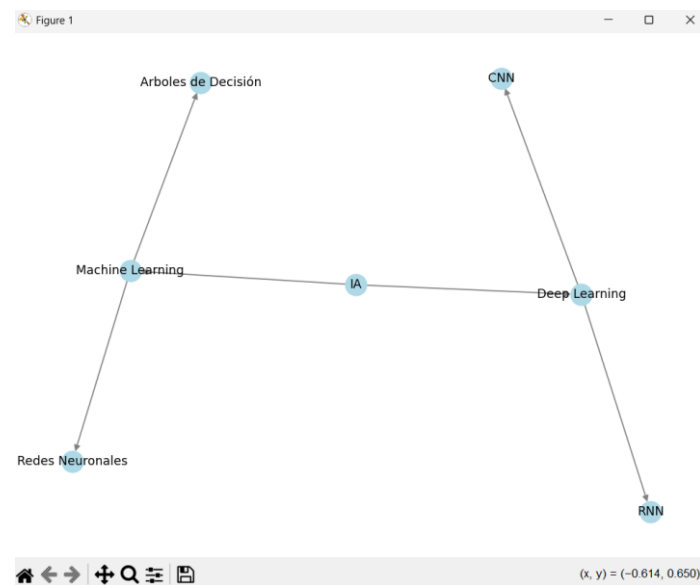
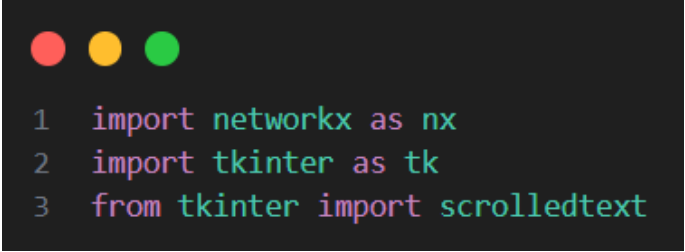


Imagen 1.4 Grafo terminado.

Chatbot

Para el chatbot se utilizaron tres librerías para programar el chatbot, las cuales fueron:

Networkx, tkinter con scrolledtext.



```
1 import networkx as nx
2 import tkinter as tk
3 from tkinter import scrolledtext
```

Imagen 2.1 Librerías para el chatbot

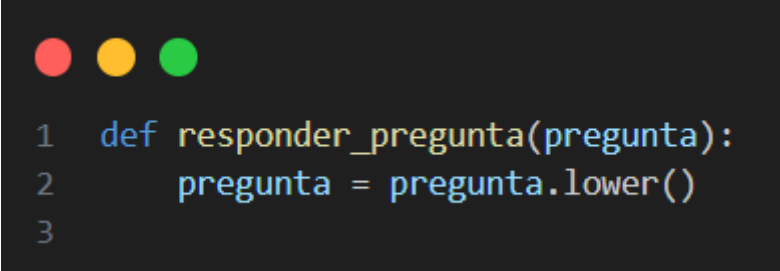
Se tomo el contexto de IA, esta vez se uso networkx como veníamos usando para crear el grafo y posteriormente se creó el nodo de los grafos.



```
1 #nodos
2 nodos=[
3     ("Inteligencia Artificial", "Machine Learning"),
4     ("Inteligencia Artificial", "Deep Learning"),
5     ("Inteligencia Artificial", "Computer Vision"),
6     ("Inteligencia Artificial", "IA Simbólica"),
7     ("Inteligencia Artificial", "Procesamiento de Lenguaje Natural"),
8
9     ("Machine Learning", "Redes Neuronales"),
10    ("Machine Learning", "Árboles de Decisión"),
11    ("Machine Learning", "Regresión"),
12    ("Machine Learning", "Clustering"),
13    ("Machine Learning", "Refuerzo"),
14
15    ("Deep Learning", "CNN"),
16    ("Deep Learning", "RNN"),
17    ("Deep Learning", "Transformers"),
18    ("Deep Learning", "GANs"),
19
20    ("IA Simbólica", "Sistemas Expertos"),
21    ("IA Simbólica", "Lógica Difusa"),
22
23    ("Computer Vision", "Reconocimiento Facial"),
24    ("Computer Vision", "Detección de Objetos"),
25    ("Computer Vision", "Segmentación de Imágenes"),
26
27    ("Procesamiento de Lenguaje Natural", "Chatbots"),
28    ("Procesamiento de Lenguaje Natural", "Traducción Automática"),
29    ("Procesamiento de Lenguaje Natural", "Análisis de Sentimientos"),
30
31 ]
```

Imagen 2.2 Nodo principal para el chatBot.

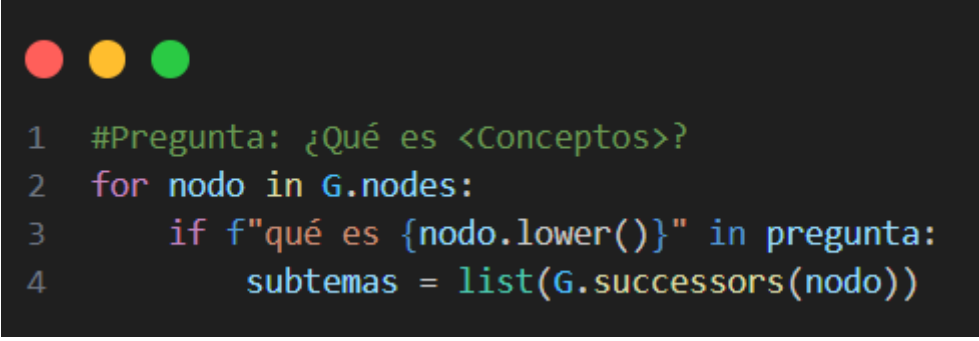
Después de eso se hicieron las preguntas mediante una función la cual analiza lo que escribe el usuario y devuelve una respuesta basada en el grafo implementado, la parte de `.lower` convierte la pregunta a minúsculas y así evitar problemas al comparar.



```
1 def responder_pregunta(pregunta):
2     pregunta = pregunta.lower()
3
```

Imagen 2.3 Función responder_pregunta.

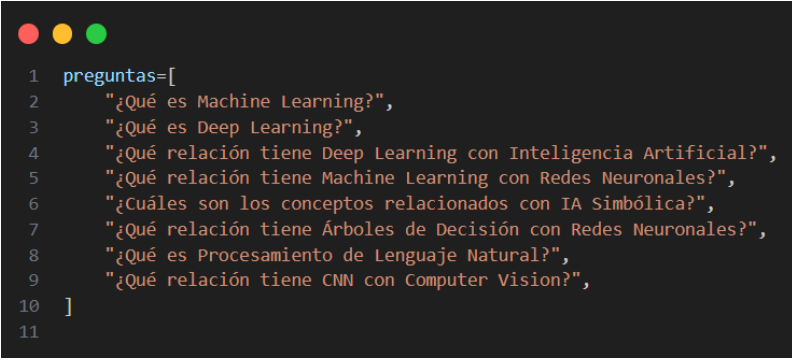
Se creo la pregunta ¿Qué es?, la cual recorrer todo el nodo del grafo y si encuentra que la pregunta esta la frase “qué es”, detecta la petición del usuario, luego los subtemas del nodo tienen relaciones.



```
1 #Pregunta: ¿Qué es <Conceptos>?
2 for nodo in G.nodes:
3     if f"qué es {nodo.lower()}" in pregunta:
4         subtemas = list(G.successors(nodo))
```

Imagen 2.4 For del nodo para las preguntas.

El código luego reconoce estas preguntas y las imprime junto a la respuesta de responder_pregunta.

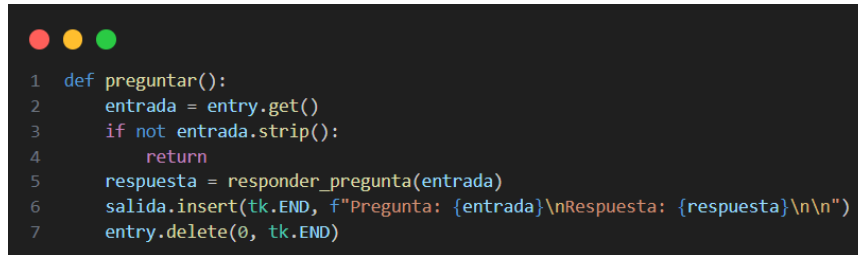


```
1 preguntas=[
2     "¿Qué es Machine Learning?",
3     "¿Qué es Deep Learning?",
4     "¿Qué relación tiene Deep Learning con Inteligencia Artificial?",
5     "¿Qué relación tiene Machine Learning con Redes Neuronales?",
6     "¿Cuáles son los conceptos relacionados con IA Simbólica?",
7     "¿Qué relación tiene Árboles de Decisión con Redes Neuronales?",
8     "¿Qué es Procesamiento de Lenguaje Natural?",
9     "¿Qué relación tiene CNN con Computer Vision?",
10 ]
11
```

Imagen 2.5 Lista de pruebas preguntas

La siguiente función se ejecuta cuando el usuario hace clic en el botón “Preguntar”.

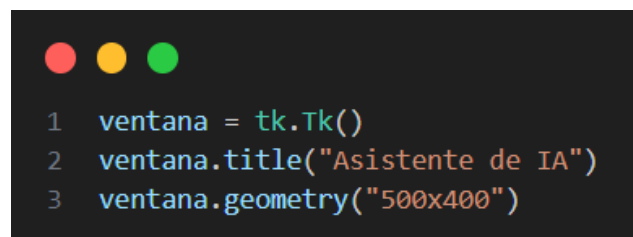
El cual toma de la caja de entrada, llama a responder_pregunta, muestra la respuesta en el área de texto que es la salida y limpia la caja de entrada para la siguiente pregunta.

A screenshot of a code editor with a dark background and three colored window control buttons (red, yellow, green) in the top left corner. The code is written in Python and defines a function named 'preguntar()'.

```
1 def preguntar():
2     entrada = entry.get()
3     if not entrada.strip():
4         return
5     respuesta = responder_pregunta(entrada)
6     salida.insert(tk.END, f"Pregunta: {entrada}\nRespuesta: {respuesta}\n\n")
7     entry.delete(0, tk.END)
```

Imagen 2.6 Función preguntar()

Ahora pasemos al a pequeña interfaz usando la librería de Tkinter. Se crea una ventana que permite interactuar con el asistente. Donde se coloca el titulo y el tamaño que tendrá la ventana.

A screenshot of a code editor with a dark background and three colored window control buttons (red, yellow, green) in the top left corner. The code is written in Python and creates a Tkinter window.

```
1 ventana = tk.Tk()
2 ventana.title("Asistente de IA")
3 ventana.geometry("500x400")
```

Imagen 2.7 Ventana con Tkinter.

Después se crearon elementos como label, para el texto inicial que indica al usuario que hacer, button para ejecutar preguntar() y un scrolledText, que es el área de texto grande con scroll donde muestra las preguntas y respuestas.

Y finalmente se mantiene la ventana abierta y esperando su interacción.

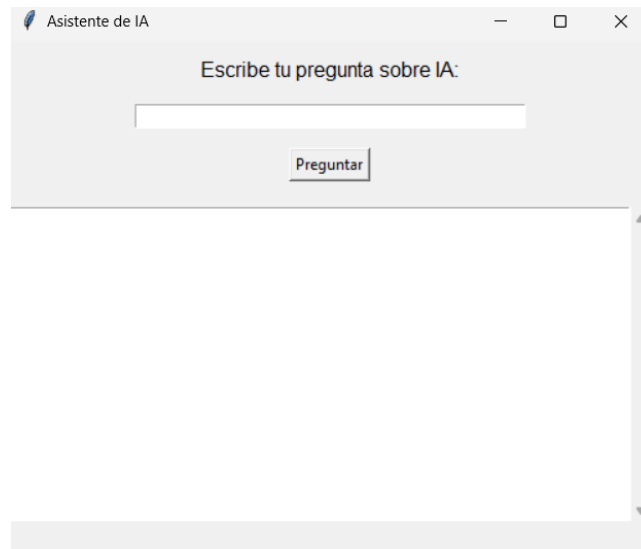


Imagen 2.8 Ventana terminada.

Así se ve la ventana con la interacción del usuario al mandar una pregunta al chatBot.

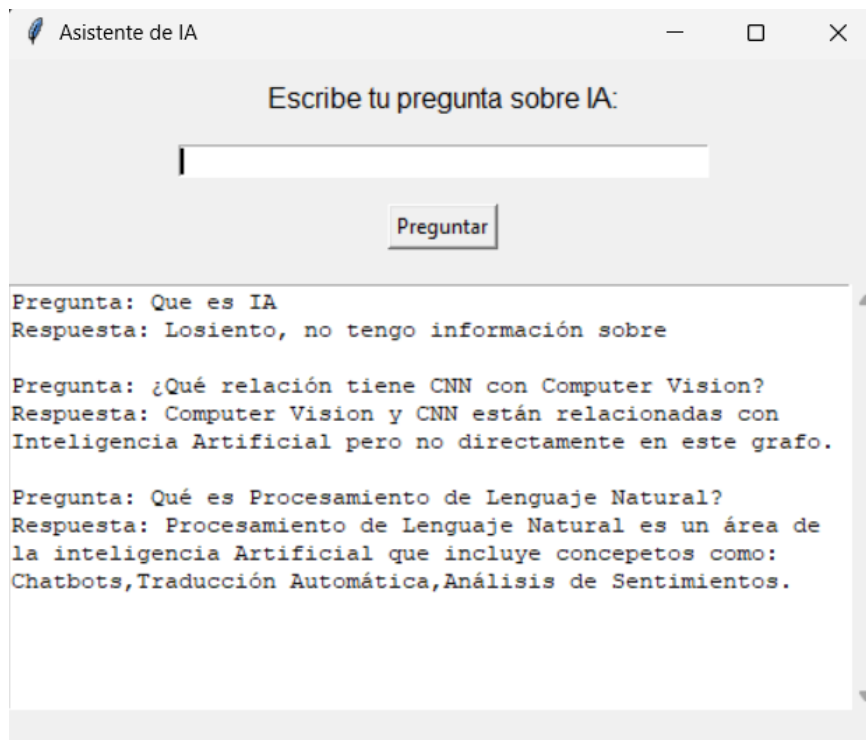


Imagen 2.9 Ventana con interacción del usuario.

Reconocimiento de voz con servicios

Para el reconocimiento de voz se utilizaron las siguientes librerías en el archivo main.py las cuales permiten el reconocimiento de voz y se agrega la ruta de la carpeta “ffmpeg” la cual se debe descargar previamente.

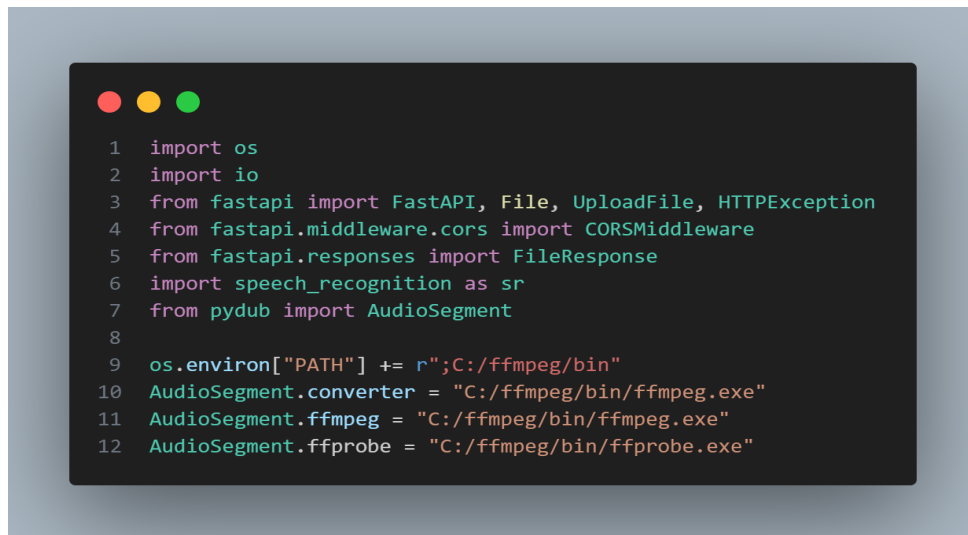


Imagen 3.1 Importaciones de librería y el archivo bin.

Posteriormente se configura los orígenes que permitirá la API, para evitar errores se coloca el “*” el cual permite cualquier origen

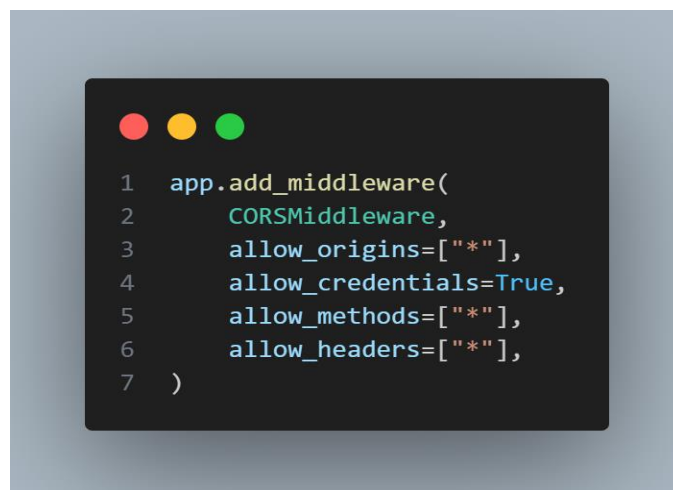


Imagen 3.2 Implementación de la API.

Ahora se va a mapear la página HTML que mostrara la información



Imagen 3.3 Mapeando al archivo Index.html

Para finalizar el archivo de Python se genera el mapeo de método “POST” el cual hará uso de la librería de “speech_recognition” la cual recibirá el audio y lo transcribirá para su posterior lectura.



Imagen 3.4 mapeo del método POST

Posteriormente en el archivo HTML se agregará un script el cual comenzará conectándose con el objeto llamado “app” y regresando los valores de la página al defecto. Posteriormente en el método “startRecording()” se dará acceso al micrófono del dispositivo y se iniciará la grabación de audio. En caso de no poder acceder al micrófono se notificará en la consola.



```
1  new Vue({
2    el: '#app',
3    data: {
4      recording: false,
5      mediaRecorder: null,
6      audioChunks: [],
7      audioBlob: null,
8      audioUrl: '',
9      resultText: ''
10   },
11   methods: {
12     startRecording() {
13       navigator.mediaDevices.getUserMedia({ audio: true })
14         .then(stream => {
15           this.mediaRecorder = new MediaRecorder(stream);
16           this.mediaRecorder.start();
17           this.recording = true;
18           this.audioChunks = [];
19           this.mediaRecorder.ondataavailable = event => {
20             if (event.data.size > 0) {
21               this.audioChunks.push(event.data);
22             }
23           };
24           this.mediaRecorder.onstop = () => {
25             this.audioBlob = new Blob(this.audioChunks, { type: 'audio/wav' });
26             this.audioUrl = URL.createObjectURL(this.audioBlob);
27           };
28         })
29         .catch(err => {
30           console.error("Error al acceder al micrófono:", err);
31         });
32     },
```

Imagen 3.5 script para iniciar la grabación de audio

Por último, se agrega el método “stopRecording()” el cual detiene la grabación y el método “sendAudio()” el cual envía el audio al back y guarda la respuesta en un archivo .json para transcribirlo a la página.



```
1  stopRecording() {
2    this.mediaRecorder.stop();
3    this.recording = false;
4  },
5  sendAudio() {
6    const formData = new FormData();
7    formData.append('audio_file', this.audioBlob, 'recording.wav');
8    console.log(this.audioBlob);
9    fetch('http://127.0.0.1:8000/recognize', {
10     method: 'POST',
11     body: formData
12   })
13     .then(response => response.json())
14     .then(data => {
15       this.resultText = data.text;
16     })
17     .catch(error => {
18       console.error("Error al enviar el audio:", error);
19     });
20   }
}
```

Imagen 3.6 script para detener y enviar el audio al back

A continuación, se muestra como se ve la página HTML finalizada

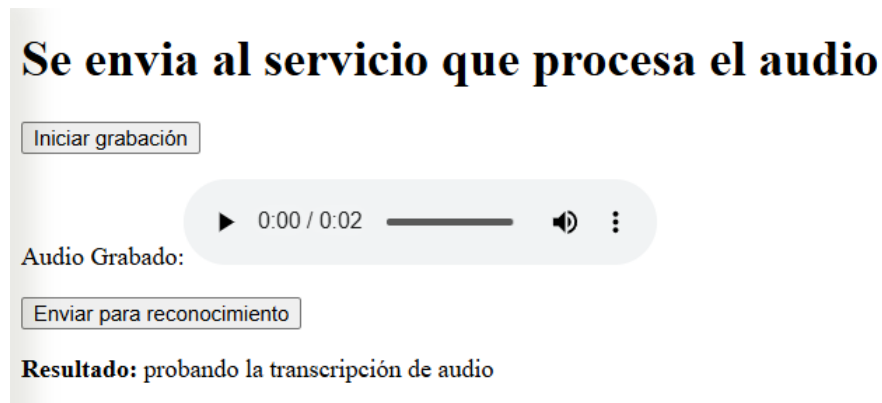


Imagen 3.7 ejecución de la página HTML

Chatbot con audio y texto

En el archivo main.py se agregarán las librerías para reconocer audio, creación de FastAPI y se agregan las rutas de la carpeta "ffmpeg" descargada anteriormente.

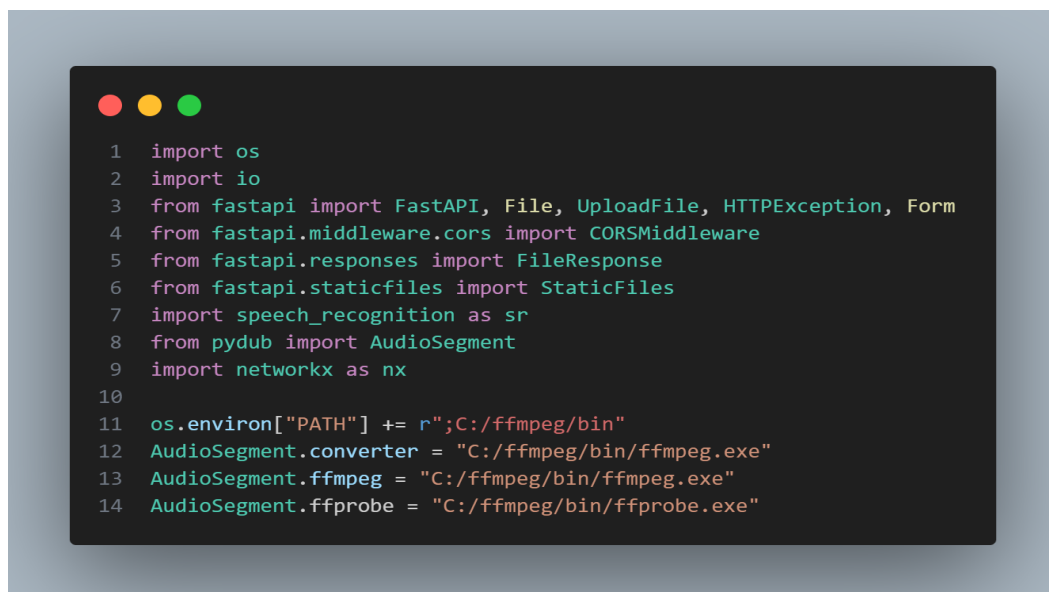


Imagen 4.1 importación de librerías

Se crea una base de conocimientos en la cual estarán todas las relaciones que tendrá la información

```
1  G = nx.DiGraph()
2
3  nodos = [
4      ("Howards", "Grifindor"),
5      ("Howards", "Slytherin"),
6      ("Howards", "Ravenclaw"),
7      ("Howards", "Hufelpuf"),
8      ("Howards", "Cáliz de fuego"),
9
10     ("Grifindor", "León"),
11     ("Grifindor", "Dumbeldor"),
12     ("Grifindor", "Harry Potter"),
13     ("Grifindor", "Ron"),
14     ("Grifindor", "Hermioni"),
15
```

Imagen 4.2 fragmento de la base de conocimientos

Una vez creada la base de conocimientos se agrega la sección de preguntas en la cual se pondrán las preguntas que podrán ser respondidas. En esta sección se recorrerán todos los nodos buscando que relación tienen entre ellos y saber si tiene subtemas.

```
1  G.add_edges_from(nodos)
2
3  def responder_pregunta(pregunta):
4      pregunta = pregunta.lower()
5
6      for nodo in G.nodes:
7          if f"que es {nodo.lower()}" in pregunta or f"qué es {nodo.lower()}" in pregunta:
8              subtemas = list(G.successors(nodo))
9              if subtemas:
10                 return f"{nodo} está relacionado con: {' '.join(subtemas)}."
11             else:
12                 return f"{nodo} es un concepto del universo de Harry Potter, pero no tiene subcategorías en este modelo."
13
14     conceptos = [nodo for nodo in G.nodes if nodo.lower() in pregunta]
15     if len(conceptos) == 2:
16         nodo1, nodo2 = conceptos
17         if G.has_edge(nodo1, nodo2):
18             return f"{nodo2} está relacionado directamente con {nodo1}."
19         elif G.has_edge(nodo2, nodo1):
20             return f"{nodo1} está relacionado directamente con {nodo2}."
21         else:
22             return f"{nodo1} y {nodo2} pertenecen al universo de Harry Potter pero no tienen relación directa en este grafo."
23
24     return "No tengo información específica sobre eso en mi base de conocimiento de Harry Potter."
```

Imagen 4.3 fragmento del código de preguntas

Se crea el mapeo de la página principal HTML



Imagen 4.4 mapeo de la página HTML

A continuación, se hace el mapeo del método POST para reconocimiento de audio



Imagen 4.5 mapeo del método POST para audio

Debajo del mapeo “POST” para reconocimiento de audio se agregará el método “POST” pero esta vez para el reconocimiento de texto el cual recibirá el texto escrito por el usuario con la pregunta, y se devolverá la respuesta generada en la sección donde se escribieron las respuestas que daría el programa.

```
1 @app.post("/pregunta-texto")
2 async def pregunta_texto(pregunta: str = Form(...)):
3     try:
4         if not pregunta.strip():
5             raise HTTPException(status_code=400, detail="La pregunta no puede estar vacía")
6
7         respuesta = responder_pregunta(pregunta)
8
9         return {
10             "pregunta": pregunta,
11             "respuesta": respuesta
12         }
13
14     except Exception as e:
15         print("Error:", str(e))
16         raise HTTPException(status_code=500, detail=str(e))
```

Imagen 4.6 mapeo del método POST para el texto

En el archivo HTML se agregará un script el cual comenzará conectándose con el objeto llamado “app” e iniciando los valores de la página en defecto.

```
1 new Vue({
2     el: '#app',
3     data: {
4         isRecording: false,
5         mediaRecorder: null,
6         audioChunks: [],
7         audioBlob: null,
8         audioURL: '',
9
10        preguntaTexto: '',
11
12        processingAudio: false,
13        processingText: false,
14
15        resultadoActual: null,
16        errorText: ''
17    },
18    methods: {
```

Imagen 4.7 inicio de la página HTML

Dentro de methods se agregará el método “startRecording()” en el cual se dará acceso al micrófono del dispositivo y se iniciará la grabación de audio. En caso de no poder acceder al micrófono se notificará en la consola.

```
1  async startRecording() {
2      if (this.isRecording) return;
3
4      try {
5          this.errorText = '';
6          const stream = await navigator.mediaDevices.getUserMedia({ audio: true });
7          this.mediaRecorder = new MediaRecorder(stream, {
8              mimeType: 'audio/webm; codecs=opus'
9          });
10         this.audioChunks = [];
11
12         this.mediaRecorder.ondataavailable = event => {
13             if (event.data.size > 0) {
14                 this.audioChunks.push(event.data);
15             }
16         };
17
18         this.mediaRecorder.onstop = () => {
19             this.audioBlob = new Blob(this.audioChunks, { type: 'audio/webm' });
20             this.audioURL = URL.createObjectURL(this.audioBlob);
21             if (this.mediaRecorder && this.mediaRecorder.stream) {
22                 this.mediaRecorder.stream.getTracks().forEach(track => track.stop());
23             }
24         };
25
26         this.mediaRecorder.start();
27         this.isRecording = true;
28     } catch (err) {
29         this.errorText = "Error al acceder al micrófono: " + err.message;
30         console.error("Error al acceder al micrófono", err);
31     }
```

Imagen 4.8 inicio de la grabación de audio

método “sendAudio()” el cual envía el audio al back y guarda la respuesta en un archivo .json para transcribirlo y solicitar la respuesta a la pregunta.

```
1  async sendAudio() {
2      try {
3          this.errorText = '';
4          this.processingAudio = true;
5
6          const formData = new FormData();
7          formData.append('audio_file', this.audioBlob, 'audio.webm');
8
9          const response = await fetch('http://127.0.0.1:8000/recognize', {
10              method: 'POST',
11              body: formData
12          });
13
14          if (!response.ok) {
15              throw new Error(Error);
16          }
17
18          const data = await response.json();
19          this.resultadoActual = {
20              pregunta: data.text,
21              respuesta: data.respuesta,
22          };
23      } catch (error) {
24          this.errorText = 'Error al enviar el audio: ' + error.message;
25          console.error('Error al enviar el audio', error);
26      } finally {
27          this.processingAudio = false;
28      }
29  },
```

Imagen 4.9 Envío del audio al back

Para enviarla pregunta en formato de texto primeramente se convierte el texto ingresado por el usuario a un formato .json para enviarlo al back y que este lo pueda procesar, una vez procesado devuelve la respuesta a dicha pregunta.

```
1  async enviarTexto() {
2      try {
3          this.errorText = '';
4          this.processingTexto = true;
5
6          const formData = new FormData();
7          formData.append('pregunta', this.preguntaTexto);
8
9          const response = await fetch('http://127.0.0.1:8000/pregunta-texto', {
10             method: 'POST',
11             body: formData
12         });
13
14         if (!response.ok) {
15             throw new Error(Error);
16         }
17
18         const data = await response.json();
19         this.resultadoActual = {
20             pregunta: data.pregunta,
21             respuesta: data.respuesta,
22         };
23
24         this.preguntaTexto = '';
25     } catch (error) {
26         this.errorText = 'Error al enviar la pregunta: ' + error.message;
27         console.error('Error al enviar la pregunta', error);
28     } finally {
29         this.processingTexto = false;
30     }
31 }
```

Imagen 4.10 Envío de pregunta en formato de texto

A continuación, se muestra la página HTML en ejecución tanto con audio como con texto:

Asistente de Attack on Titan

Escribe tu pregunta:

Enviar pregunta

O haz tu pregunta por voz:

Mantén presionado para grabar

Audio grabado:

▶ 0:00 / 0:02 🔊 ⋮

Enviar audio

Tu pregunta: "Qué es cáliz de fuego"

Respuesta: Cáliz de fuego está relacionado con: Torneo de los tres magos, Voldemort, Harry Potter, Cedric, Dragones, Lago, Laberinto.

Asistente de Attack on Titan

Escribe tu pregunta:

Enviar pregunta

O haz tu pregunta por voz:

Mantén presionado para grabar

Audio grabado:

▶ 0:00 / 0:02 🔊 ⋮

Enviar audio

Tu pregunta: "que es howards"

Respuesta: Howards está relacionado con: Grifindor, Slytherin, Ravenclaw, Hufelpuf, Cáliz de fuego.

Conclusiones

El uso de grafos para representar un modelo de relaciones en IA es primordial si se requiere ver sus relaciones. Además, la implementación de chatbots junto con el reconocimiento de voz, nos da la posibilidad de desarrollar sistemas interactivos más dinámicos. La integración de texto y audio en los chats bots representa algo más dinámico.

Entonces podemos decir que la inteligencia artificial con la implementando librerías que le ayuden a darle esa interacción de aprendizaje puede mostrar todo ese mecanismo interactivo con el usuario, como lo mostramos en estos ejemplos sencillos dándole una base de conocimiento fija puede llegar a darle al usuario un resultado esperado.